Technika obrazowa – Projekt

Projekt z przedmiotu TO realizowany jest w trybie konsultacyjnym. Projekty prowadzą:

- dr inż Andrzej Buchowicz, pokój 451, andrzej.buchowicz@pw.edu.pl
- dr inż Grzegorz Galiński, pokój 451, grzegorz.galinski@pw.edu.pl

Terminy konsultacji dostępne są na stronie IRiTM (https://www.ire.pw.edu.pl/struktura/nauczyciele-akademiccy/), ponadto można kontaktować się za pośrednictwem poczty elektronicznej oraz platformy MS Teams.

Projekt realizowany jest w zespołach, preferowane zespoły 4-osobowe.

Przydział zespołów do konkretnego prowadzącego zostanie dokonany po ustaleniu wszystkich zespołów - od tego momentu należy kontaktować się z właściwym prowadzącym.

Realizacja projektu jest podzielona na etapy (patrz: Harmonogram realizacji projektu), których zaliczenie jest obowiązkowe - wymagany jest kontakt z prowadzącym oraz sporządzenie krótkiego sprawozdania.
Należy samodzielnie pilnować terminów zaliczania etapów podanych w harmonogramie projektu. Niezaliczenie etapu w określonym terminie będzie skutkowało obniżeniem oceny końcowej z projektu.

Do zaliczenia projektu wymagane jest przedstawienie:

- sprawozdania zawierającego opis realizacji zadania, wyniki przeprowadzonych testów, wnioski, a także **podział zadań / udział poszczególnych osób w realizacji projektu**;
- kodu źródłowego aplikacji (udostępnienie przez repozytorium `git`);
- zbioru danych testowych.

Harmonogram realizacji projektu:

1. Zapis do grup projektowych (zespoły czteroosobowe) – do 18 X.
2. Wstępne wyniki implementacji/testowania – do 13 XII (wymagany kontakt z prowadzącym).
3. Zakończenie projektu: sprawozdanie, wyniki testów, wnioski, kody źródłowe - do 29 I (wymagany kontakt/spotkanie z prowadzącym).

Tematy projektów:

1. Detekcja twarzy z wykorzystaniem splotowych sieci neuronowych
   - wykorzystać modele sieci neuronowych dostępne w bibliotece PyTorch/TorchVision (https://pytorch.org/vision/stable/models.html#classification);
   - metodą uczenia transferowego (*transfer learning*)/dostrajania (*finetuning*) dostosować model sieci do detekcji dwóch klas obiektów: twarz i nie-twarz (tło) (https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html, https://pytorch.org/tutorials/intermediate/torchvision_tutorial.html);
   - do uczenia i testowania sieci wykorzystać bazę obrazów WIDER (http://shuoyang1213.me/WIDERFACE/), z dostępnych obrazów należy wyciąć fragmenty zawierające twarz (na podstawie dostępnego opisu), fragmenty powinny zawierać losowej wielkości margines wokół twarzy (w tym zerowy oraz „ujemny"), należy je przeskalować do jednego, określonego rozmiaru; podobnie trzeba przygotować fragmenty nie zawierające twarzy (przykłady tła); można również skorzystać ze zbioru LFWPeople (https://www.kaggle.com/datasets/atulanandjha/lfwpeople), innego ogólnodostępnego zbioru lub własnych zdjęć;

UWAGA: bazy obrazów dostępne są również w bibliotece PyTorch/TorchVision (https://pytorch.org/vision/main/datasets.html);
- sprawdzić działanie i efektywność kilku dostępnych modeli;
- działanie sieci sprawdzić również na obrazach zawierających wiele twarzy, np. metodą przesuwnego okna o różnym rozmiarze.

2. Badanie wpływu kwantyzacji współczynników DCT na jakość obrazu
- obliczyć DCT, dokonać kwantyzacji współczynników transformaty i zrekonstruować obraz;
- dwa tryby obliczania DCT: w blokach NxN pikseli oraz dla całego obrazu;
- współczynniki kwantyzacji określane są w programie (GUI), każdy z nich może być modyfikowany niezależnie;
- sprawdzić jak „siła" kwantyzacji poszczególnych współczynników wpływa na jakość obrazu zrekonstruowanego (kwantyzacja jednego, wybranego współczynnika z różnym krokiem kwantyzacji): ocena subiektywna (widoczność zniekształceń), miary jakości np. PSNR, SSIM; w trybie blokowym dla wszystkich współczynników DCT, w trybie pełnoobrazowym dla wybranych współczynników.

3. Kompresja obrazu z wykorzystaniem kodowania różnicowego/predykcyjnego
- różne tryby predykcji (1, 2, 3 sąsiednie piksele, LPC z wykorzystaniem różnej długości bloków);
- porównanie danych oryginalnych i uzyskanych danych różnicowych: histogramy wartości, energia, entropia;
- zbudowanie kodu Huffmana i porównanie jego efektywności z wyliczoną entropią.

4. Stabilizacja sekwencji wideo metodą globalnego wektora ruchu
- znaleźć przesunięcie (x, y) dające najmniejszy błąd pomiędzy dwiema kolejnymi ramkami sekwencji wideo;
- dokonać transformacji (przesunięcia) ramki w celu usunięcia ruchu kamery (nieruchome obiekty sceny wyświetlane są dokładnie w tym samym miejscu);
- sprawdzić działanie metody dla miar (MAE, MSE, …) dla różnych przestrzeni kolorów (RGB, HSV, YCbCr/YUV, …);
- zastosować hierarchiczne wyznaczanie wektora ruchu w celu zmniejszenia złożoności obliczeniowej.

5. Detekcja ruchu w sekwencji wideo
- wykorzystać:
  a) różnicę pomiędzy obrazami do detekcji obiektów ruchomych,
  b) narzędzia „foreground extraction"/„background subtraction" dostępne w bibliotece openCV (https://docs.opencv.org/4.5.3/d1/dc5/tutorial_background_subtraction.html),
  c) sieci neuronowe;
- segmentacja, filtracja w celu usunięcia szumu, określenie pozycji obiektów pierwszoplanowych;
- zgłoszenie alarmu w przypadku wykrycia ruchu, zaznaczenie obiektów ruchomych na obrazie, wyświetlenie obrazu;
- sprawdzenie działania w różnych przestrzeniach barw (RGB, HSV, YCbCr/YUV, …).

6. Śledzenie obiektów w sekwencji wideo
- wykorzystać:
  a) narzędzia dostępne w bibliotece openCV, np. algorytmy MEANSHIFT, CAMSHIFT (https://docs.opencv.org/4.5.3/d7/d00/tutorial_meanshift.html) i inne,
  b) sieci neuronowe;
- śledzony obiekt określany w programie (GUI);

- określenie przesunięcia śledzonego obiektu, zaznaczenie na obrazie pozycji obiektu wraz z wektorem jego przesunięcia, wyświetlenie obrazu;
- budowa i prezentacja „ścieżki" obiektu: położenie wybranego punktu lub punktów we wszystkich wcześniejszych obrazach w sekwencji (np. centroid albo wierzchołki prostokąta otaczającego obiekt);
- sprawdzenie działania w różnych przestrzeniach barw (RGB, HSV, YCbCr/YUV, …);
- sprawdzenie wpływu filtracji wyników propagacji wstecznej (ang. *back propagation*).

7. Wyznaczanie mapy dysparycji
   - korzystając z metod głębokiego uczenia (przedstawionych m.in. w https://learnopencv.com/disparity-estimation-using-deep-learning/) wyznaczyć mapę dysparycji dla wybranych par obrazów stereowizyjnych dostępnych m.in. w zbiorach testowych:
     - KITTY (https://www.cvlibs.net/datasets/kitti/eval_stereo.php),
     - Middlebury Stereo Datasets 2005 (https://vision.middlebury.edu/stereo/data/scenes2005/),
     - High Resolution Stereo Datasets 2014 (https://vision.middlebury.edu/stereo/data/scenes2014/);
   - ocenić wpływ parametrów przekazywanych do algorytmu wyznaczającego mapę dysparycji na 'jakość' mapy dysparycji przez porównanie map wyznaczonych przez algorytm z mapami referencyjnymi (*ground truth*).

8. Łączenie obrazów w panoramę
   - korzystając z narzędzi dostępnych w bibliotece OpenCV (https://www.pyimagesearch.com/2018/12/17/image-stitching-with-opencv-and-python/, https://kushalvyas.github.io/stitching.html) opracuj aplikację umożliwiającą łączenie obrazów w panoramę;
   - przetestuj opracowaną aplikację na wybranych obrazach testowych, np. pochodzących ze zbioru Adobe Panoramas Dataset (http://sourceforge.net/projects/adobedatasets.adobe/files/adobe_panoramas.tgz/download).

9. Zwiększanie rozdzielczości obrazów
   - korzystając z narzędzi wykorzystujących techniki uczenia głębokiego, przedstawione m.in. w:
     - https://pyimagesearch.com/2024/04/29/unlocking-image-clarity-a-comprehensive-guide-to-super-resolution-techniques/,
     - https://pyimagesearch.com/2024/06/03/sharpen-your-vision-super-resolution-of-cctv-images-using-hugging-face-diffusers/

     opracuj aplikację umożliwiającą powiększanie rozdzielczości obrazów;
   - przetestuj opracowaną aplikację na wybranych obrazach testowych, porównaj 'jakość' obrazów uzyskanych przy pomocy opracowanej aplikacji z obrazami, których rozdzielczość zwiększono stosując klasyczne techniki interpolacji.

10. Rozpoznawanie tekstu przy użyciu metod uczenia głębokiego
    - opracować aplikację umożliwiającą rozpoznawanie testu w obrazach korzystając z narzędzi przedstawionych m.in. w:
      - https://learnopencv.com/trocr-getting-started-with-transformer-based-ocr/
      - https://github.com/tesseract-ocr/tesseract
    - przygotować zestaw obrazów testowych zawierających teksty, np. zdjęć dokumentów wykonanych aparatem fotograficznym w telefonie komórkowych, obrazów dokumentów uzyskanych przy użyciu skanera, itp. Oszacować wpływ jakości obrazów na dokładność rozpoznawania tekstu.

11. Klasyfikacja obrazów z wykorzystaniem splotowych sieci neuronowych i uczenia transferowego
    - zbiór danych: Food-101 (https://huggingface.co/datasets/ethz/food101, https://docs.pytorch.org/vision/main/generated/torchvision.datasets.Food101.html);
    - charakterystyka zbioru danych: liczba klas, liczba obrazów w poszczególnych klasach, rozdzielczość, format, model koloru itd., wizualizacja wybranych obrazów;
    - podział zbioru danych na podzbiory: trenujący, walidujący, testowy;
    - przygotowanie zbioru danych do trenowania: zmiana rozdzielczości, normalizacja wartości średniej i odchylenia standardowego, wybór metody augmentacji danych;
    - model bazowy:
        - wytrenować na podzbiorze trenującym prostą sieć splotową:
            - dane wejściowe: obrazy RGB o rozdzielczości 224x224 pikseli,
            - ekstrakcja cech: np. 2..4 warstwy Conv + ReLU + MaxPool2D,
            - klasyfikator: FFNN + softmax;
        - wyznaczyć miary opisujące efektywność sieci na podzbiorze testowym;
    - uczenie transferowe (*transfer learning*):
        - wybrać model sieci splotowej wytrenowanej na zbiorze ImageNet (np. ResNet18, Resnet50),
        - zamrozić wszystkie warstwy splotowe,
        - zmodyfikować warstwę wyjściową/klasyfikator tak, aby liczba wyjść była zgodna z liczbą klas w zbiorze danych,
        - wytrenować sieć na podzbiorze trenującym,
        - przeprowadzić analizę procesu trenowania sieci:
            - wykresy: wartość funkcji kosztu/dokładność dla podzbioru trenującego i walidującego w kolejnych epokach,
            - sprawdzić czy nie występuje przetrenowanie (*overfitting*), niedouczenie (*underfitting*) sieci;
        - wyznaczyć miary opisujące efektywność sieci na zbiorze testowym, porównać wyniki z wynikami uzyskanymi dla modelu bazowego,
        - powtórzyć trenowanie i ocenę efektywności przy zmienionych wartościach hiperparametrów, m.in. rozmiar wsadu (*batch size*), współczynnik uczenia (*learning rate*); ocenić wpływ wartości hiperparametrów na przebieg procesu uczenia i efektywność wytrenowanego modelu;
    - dostrajanie (*fine tuning*):
        - w modelu wybranym do uczenia transferowanego zablokować początkowe warstwy splotowe, pozostawić możliwość modyfikacji *k* ostatnich warstw splotowych,
        - zmodyfikować klasyfikator: liczba wyjść musi być równa liczbie klas w zbiorze danych,
        - wytrenować model, wyznaczyć miary opisujące efektywność,
        - powtórzyć trenowanie przy zmienionych wartościach hiperparametrów,
        - porównać wyniki z wynikami uzyskanymi dla modelu bazowego i modelu z uczeniem transferowym;
    - dla modelu, który zapewnił najlepsze wyniki przeprowadzić analizę błędów klasyfikacji: wyznaczyć macierz pomyłek, znaleźć reprezentatywne przykłady poprawnej i błędnej klasyfikacji, określić jakie cechy obrazów mogą powodować ich niepoprawną klasyfikację.

12. Klasyfikacja obrazów z wykorzystaniem splotowych sieci neuronowych i uczenia transferowego (2)
    - zbiór danych: Oxford Flower 102 (https://www.kaggle.com/datasets/demonplus/flower-dataset-102, https://docs.pytorch.org/vision/main/generated/torchvision.datasets.Flowers102.html);
    - szczegółowe zadania jak w p.10.

13. Możliwa jest również realizacja własnego tematu po uzgodnieniu z prowadzącymi.